NodeControl

NodeControl is a self-hosted B2B web platform for safely managing and executing Ansible automation through a professional web interface.

The project is designed for IT service providers, managed service providers, system houses, and internal IT teams that want to run Ansible workflows without giving every operator direct terminal, SSH, or full Ansible access.

NodeControl is not intended to replace Ansible. It acts as a control plane around Ansible: customer separation, permissions, inventory management, playbook execution, scheduled jobs, job history, logs, and auditability.

Product Terminology

The backend keeps the pragmatic domain names used by the execution model, while the frontend presents product-facing labels:

- `Job` is shown as **Action**.
- `JobRun` is shown as **Run**.
- `ManagedNode` is shown as **Host**.
- `ControlNode` is shown as **Control Host**.
- `InventoryGroup` is shown as **Inventar**.
- `Customer` is shown as **Kunde** where the UI uses German labels.

Product Goal

NodeControl should become a product-grade portfolio project and a realistic self-hosted B2B product.

The main value proposition is:

> Run Ansible workflows safely, repeatedly, and auditably across customer environments without direct terminal work.

Current Capabilities

NodeControl is implemented as a working dev/demo product, not just a skeleton. The current MVP is a customer-scoped automation control plane with a complete local demo loop. Current capabilities include:

- Customer management
- User profiles, Fake Auth/OIDC authentication, and customer memberships
- Internal NodeControl roles and permissions
- Control nodes
- Managed nodes
- Inventory groups
- Inline YAML and managed artifact-directory playbooks
- Customer-scoped Git repository sources for one-time artifact import into managed playbooks/templates
- Variable sets
- Actions and manual Runs
- Scheduled Runs / cron jobs through Worker polling
- Job run history
- Job logs
- Worker-side execution diagnostics for common SSH, dispatch, workspace, and Ansible failure classes
- Run-bound Control Hosts with Worker-side dispatch preparation, local/dev execution fallback, and SSH remote dispatch
- Audit logs
- Hostzustand / TCP reachability checks processed by the Worker
- Templates as managed text resources that can be materialized into Worker run workspaces through Actions
- Secrets as protected metadata with safe `secret://...` reference validation and Worker-side execution resolution
- User overview for platform admins
- Run wizard and Run Center demo flow
- Docker-based local dev infrastructure and shell scripts for bootstrap

Supporting MVP surfaces have intentionally narrow scope:

- Templates are materialized as configured run workspace files only; NodeControl does not provide a full template
  orchestration or remote upload system.
- Secrets expose protected metadata and safe `secret://...` references through the API. Secret values are not returned
  and are resolved only by the Worker while preparing execution artifacts. API and Worker share the same ASP.NET Data
  Protection application name and key ring so values protected through the API can be unprotected by the Worker.
- The platform admin user overview is not a full identity-provider administration system.

Important current boundaries:

- The API never executes Ansible, shell commands, SSH checks, or TCP checks.
- `NodeControl.Worker` is responsible for queued Run execution, schedule polling, host connection checks, workspaces, logs, and Ansible process execution.
- Runs snapshot the selected Control Host at queue time; the Worker prepares control-host-scoped workspaces and dispatch manifests before local/dev or SSH remote Ansible execution.
- `deploy/` is dev/demo guidance only; production packaging is not complete.
- Continuous Git-backed playbooks/sync, cloud integrations, notifications, advanced secret runtime integration, and broader system/security administration are Post-MVP directions.

See `docs/MVP_BOUNDARY.md` for the explicit current-vs-Post-MVP boundary.

Target Users

Primary users:

- IT service providers
- Managed service providers
- System houses
- Internal IT departments
- Administrators who already use Ansible but need a safer UI-based workflow

Non-Goals

NodeControl is intentionally not:

- A full AWX clone
- A replacement for Ansible
- A microservice experiment
- A Keycloak-only application
- A Kubernetes-first product
- A workflow engine for every possible automation technology
- A platform that rebuilds every Ansible feature as a custom UI feature

High-Level Architecture

```
Browser
  |
  | HTTPS
  v
Next.js Web UI
  |
  | /api/v1
  v
ASP.NET Core API
  |
  | PostgreSQL
  v
NodeControl Database
  |
  | queued JobRuns / schedules
  v
NodeControl Worker
  |
  | local/dev ansible-playbook or SSH remote dispatch
  v
Control Host
  |
  | ansible-playbook + SSH inventory
  v
Managed Nodes
```

External identity providers are integrated through OIDC. Keycloak may be used as a local development/demo provider, but it is not a required product dependency.

Security and Execution Boundary

NodeControl separates configuration from operational execution.

The API is responsible for authentication, authorization, validation, persistence, and queueing Runs. It does not execute Ansible, SSH commands, shell commands, TCP checks, or system processes.

Operational execution is handled by `NodeControl.Worker`. The Worker processes queued Runs, creates workspaces, resolves protected Secrets, stages files on the selected Control Host, starts `ansible-playbook`, collects logs, and updates Run status.

Secrets are stored as protected values. Secret values are not returned by the API and are only resolved by the Worker during execution preparation.

Technology Direction

Backend:

- C#
- ASP.NET Core Minimal APIs
- Entity Framework Core
- PostgreSQL
- Database-backed Worker schedule polling
- Background worker for execution
- xUnit and integration tests

Frontend:

- Next.js
- React
- TypeScript
- MUI
- TanStack Query
- React Hook Form
- Zod

Deployment:

- Docker Compose for local dev/demo infrastructure
- Production packaging is Post-MVP; Kubernetes/Helm are not part of the MVP

Automation Concept

NodeControl models automation through Actions, Runs, Schedules, Inventories, Playbooks, Variables, Templates, and Secrets.

An Action defines what should be executed. A Run is one concrete execution of an Action. A Schedule can queue Runs automatically through a cron expression. Inventories define the Managed Hosts targeted by Ansible. Playbooks define the automation logic. Variable Sets and Templates provide reusable execution inputs.

For remote execution, the intended flow is:

```
NodeControl Worker -> Control Host -> Managed Hosts
```

The Worker connects to the selected Control Host, stages the run workspace there, and starts ansible-playbook on that Control Host. The Control Host then connects to the Managed Hosts from the inventory.

Local Dev/Demo Quick Start

Prerequisites:

NodeControl needs a few tools that are not stored inside the repository. Install them before running the project locally.

Required on the development machine:

- Git
- .NET SDK 10
- Node.js and npm
- Docker
- Docker Compose support
- OpenSSH client
- Bash-compatible shell

Required for real Ansible execution:

- Ansible on the system where `ansible-playbook` is executed:
  - on the local Worker machine for local/dev fallback execution
  - on the selected Control Host for remote Control Host execution
- OpenSSH server on Control Hosts and Managed Hosts
- Python 3 on Control Hosts and Managed Hosts
- SSH key-based access from the NodeControl Worker machine to the Control Host
- SSH key-based access from the Control Host to the Managed Hosts

On Debian/Ubuntu-based systems, install the common dependencies with:

```
sudo apt update
sudo apt install git openssh-client openssh-server python3 ansible docker.io docker-compos
```

If the .NET SDK is available through your configured package sources, install it with:

```
sudo apt install dotnet-sdk-10.0
```

If `dotnet` is still not found afterwards, install the .NET SDK through the official Microsoft package feed for your distribution and then verify it with:

```
dotnet --info
```

Enable and start Docker on Linux:

```
sudo systemctl enable --now docker
```

If your user should run Docker without `sudo`, add the user to the Docker group and log out/in again:

```
sudo usermod -aG docker "$USER"
```

First-time project restore:

```
cd src/backend
dotnet restore NodeControl.slnx
cd ../..
```

Alternatively, from the repository root:

```
dotnet restore src/backend/NodeControl.slnx
```

Restore the local .NET tools:

```
dotnet tool restore
```

Install the frontend dependencies:

```
cd src/frontend/nodecontrol-web
npm install
cd ../../..
```

Important backend solution note:

```
src/backend/NodeControl.slnx
```

The backend intentionally uses `NodeControl.slnx`. There is no `.sln` file.

Start the local PostgreSQL and Keycloak development services:

```
./scripts/dev-up.sh
```

Apply the existing EF Core migrations to the local PostgreSQL database:

```
./scripts/dev-migrate.sh
```

Start the API in one terminal and load the optional showcase data from another terminal:

```
./scripts/dev-run-api.sh
./scripts/dev-seed-demo.sh
```

With the API still running, start the Worker and frontend in separate terminals:

```
./scripts/dev-run-worker.sh
./scripts/dev-run-frontend.sh
```

Open `http://localhost:3000`. In the default Development configuration, the API uses Fake Auth and signs you in as `Dev Admin`.

For a meaningful demo, start the infrastructure, apply migrations, and run all three app processes: API, Worker, and frontend. Without the Worker, Runs, scheduled Runs, and Hostzustand checks can be queued but will not be processed.

Development stores the shared ASP.NET Data Protection key ring in `.nodecontrol/data-protection-keys` from the
repository root. Keep this directory intact across API/Worker restarts if you want existing protected Secrets to remain
decryptable. Secrets created before the shared key-ring configuration may need to be rotated or recreated.

Useful local URLs:

- Frontend: `http://localhost:3000`
- API: `http://localhost:5257`
- Current user check: `http://localhost:5257/api/v1/me`
- Keycloak dev provider: `http://localhost:18080`

Run the local validation and smoke checks:

```
./scripts/dev-smoke.sh
```

Stop the local infrastructure when you are done:

```
./scripts/dev-down.sh
```

`dev-down.sh` preserves Docker volumes. If you need to reset the database, use Docker Compose manually and be explicit about removing volumes.

Dev/Demo Scripts

The `scripts/` directory contains small shell scripts for repeatable local workflows:

- `dev-up.sh` starts the Docker Compose development infrastructure.
- `dev-down.sh` stops the development infrastructure without deleting volumes.
- `dev-migrate.sh` runs `dotnet ef database update` against `NodeControlDbContext`.
- `dev-run-api.sh` starts the API in Development mode on `http://localhost:5257`.
- `dev-run-worker.sh` starts the Worker in Development mode.
- `dev-run-frontend.sh` starts the Next.js dev server and points it at the local API.
- `dev-seed-demo.sh` creates or updates the Acme showcase data through the running API.
- `dev-smoke.sh` runs backend restore/build/test, frontend lint/build, the API execution-boundary grep, and optional HTTP checks when local services are running.

The scripts use these defaults and can be overridden through environment variables:

- `NODECONTROL_CONNECTION_STRING`
- `NODECONTROL_API_URL`
- `NODECONTROL_FRONTEND_URL`
- `NODECONTROL_API_ORIGIN`
- `ASPNETCORE_URLS`

`deploy/` currently contains dev/demo notes only. It is not a production deployment package.

Demo Story

A compact reviewer/demo path is:

1. Start local infrastructure with `./scripts/dev-up.sh`.
2. Apply migrations with `./scripts/dev-migrate.sh`.
3. Start the API with `./scripts/dev-run-api.sh`.
4. Load the showcase data with `./scripts/dev-seed-demo.sh`.
5. Run the Worker and frontend with `./scripts/dev-run-worker.sh` and `./scripts/dev-run-frontend.sh`.
6. Open `http://localhost:3000` and sign in through Fake Auth as Dev Admin.
7. Open the `Acme Managed Services` customer and inspect Hosts, Inventar, Playbooks, Variables, Actions, Schedules, Runs, Hostzustand, Users, and Audit.
8. Start the `Demo - Inventory Echo` Action through the UI or queue it with `./scripts/dev-seed-demo.sh --queue-run`, then inspect the Run Center and logs.
9. Point out customer scoping, internal roles, audit history, and the API/Worker execution boundary.

The seeded showcase is intentionally honest: the demo Action uses a safe inline Ansible debug playbook through the real Worker execution path. It does not fake successful SSH access to demo hosts. See `docs/SHOWCASE.md` for the full walkthrough.

Development Strategy

Development is performed in vertical slices.

A vertical slice means that one user-visible feature is implemented end-to-end across:

- Domain model
- Application service
- Persistence
- API endpoint
- Frontend screen
- Authorization
- Audit behavior
- Tests

Avoid creating speculative abstractions or empty folders before they are needed.

Implemented Slice Themes

The repository has moved beyond the initial skeleton. Implemented slices cover:

- Backend solution, frontend app, Docker dev infrastructure, and documentation
- Auth/current user, customers, memberships, roles, and permissions
- Nodes, inventory groups, playbooks, variable sets, actions, runs, schedules, logs, and audit
- Templates, secrets metadata/reference validation, users, Hostzustand, run wizard, and Run Center
- Frontend stability, demo surface hardening, and local dev/demo bootstrap scripts

Repository Layout

```
nodecontrol/
├── AGENTS.md
├── README.md
├── docs/
├── src/
│   ├── backend/
│   └── frontend/
├── tests/
├── deploy/
└── scripts/
```

Current Status

NodeControl is implemented through small vertical slices. The current repository includes backend, Worker, frontend, tests, EF Core migrations, Docker-based local infrastructure, and dev/demo bootstrap scripts.

Demo Variables

```
file_name: nodecontrol-message.txt
target_dir: "{{ ansible_env.HOME }}"
message_title: "Hallo vom NodeControl Playbook!"
message_body: |
  Diese Nachricht wurde auf das Zielsystem geschrieben.
  Sie kommt aus einem Template und einem Variablensatz.
file_mode: "0644"
extra_note: "Demo-Lauf aus NodeControl"
```

Demo Template

```
{{ message_title }}

{{ message_body }}

Host: {{ inventory_hostname }}
Zeitpunkt: {% if ansible_date_time is defined %}{{ ansible_date_time.iso8601 }}{% else %}u

{% if extra_note is defined and extra_note %}
Hinweis: {{ extra_note }}
{% endif %}
```

Demo Playbook

```
---
- name: Write templated message to home folder
  hosts: all
  gather_facts: true
  tasks:
    - name: Render message file from template
      ansible.builtin.template:
        src: templates/message-file.j2
        dest: "{{ target_dir | default(ansible_env.HOME) }}/{{ file_name | default('nodeco
        mode: "{{ file_mode | default('0644') }}"
```

Template Mapping - Workspace Path

templates/message-file.j2